```
In [3]: wines.head()
```

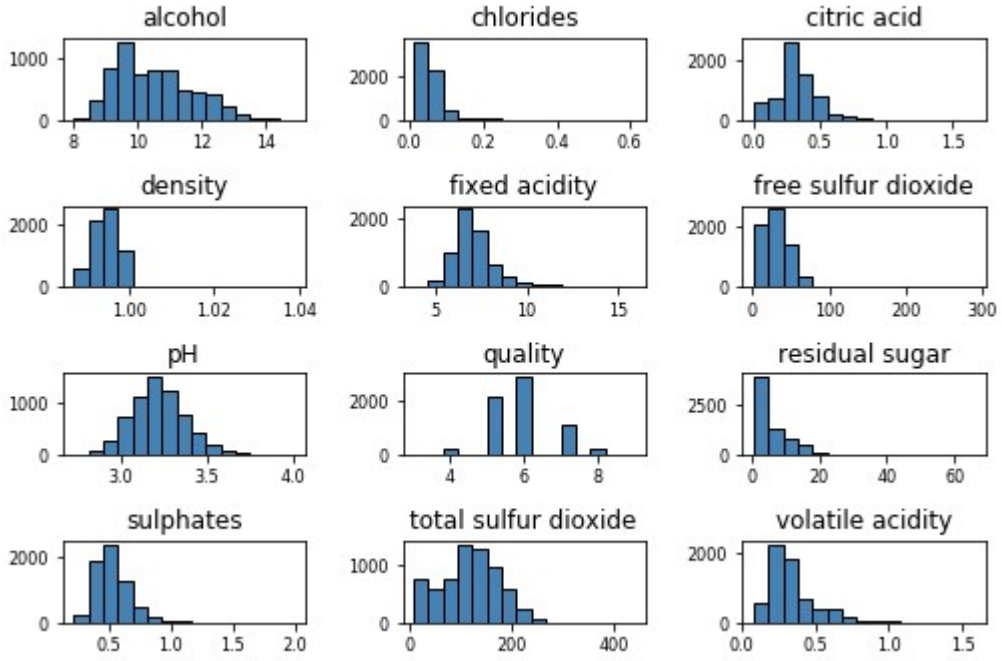
Out[3]:

| | fixed acidity | volatile acidity | citric acid | residual sugar | chlorides | free sulfur dioxide | total sulfur dioxide | density | pH | sulphates | alcohol | quality | wine_type | quality_label |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 7.0 | 0.17 | 0.74 | 12.8 | 0.045 | 24.0 | 126.0 | 0.99420 | 3.26 | 0.38 | 12.2 | 8 | white | high |
| 1 | 7.7 | 0.64 | 0.21 | 2.2 | 0.077 | 32.0 | 133.0 | 0.99560 | 3.27 | 0.45 | 9.9 | 5 | red | low |
| 2 | 6.8 | 0.39 | 0.34 | 7.4 | 0.020 | 38.0 | 133.0 | 0.99212 | 3.18 | 0.44 | 12.0 | 7 | white | medium |
| 3 | 6.3 | 0.28 | 0.47 | 11.2 | 0.040 | 61.0 | 183.0 | 0.99592 | 3.12 | 0.51 | 9.5 | 6 | white | medium |
| 4 | 7.4 | 0.35 | 0.20 | 13.9 | 0.054 | 63.0 | 229.0 | 0.99888 | 3.11 | 0.50 | 8.9 | 6 | white | medium |

The wine quality dataset

# Visualizing data in One Dimension (1-D)

## *Histograms*

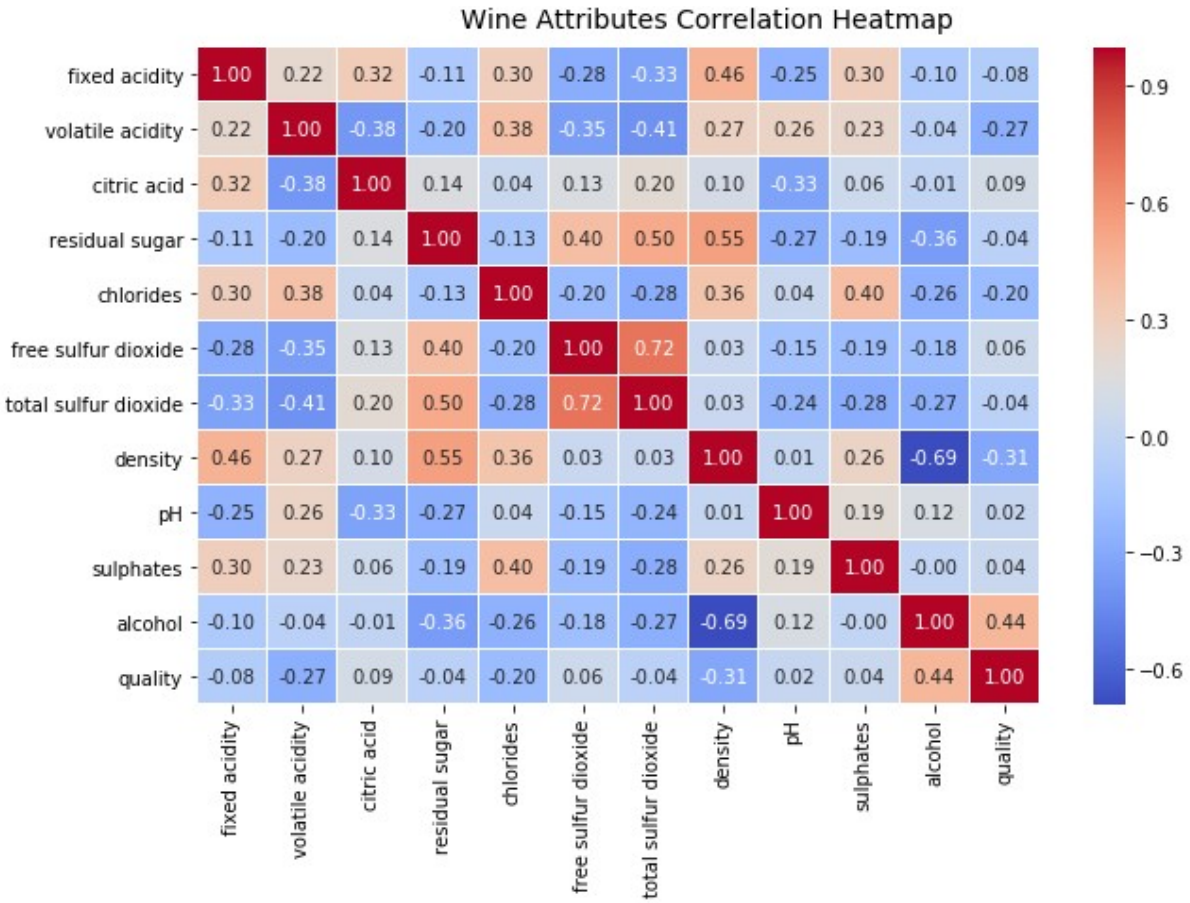



**Visualizing attributes as one-dimensional data**

The plots above give a good idea about the basic data distribution of any of the attributes.

## Visualizing data in Two Dimensions (2-D)

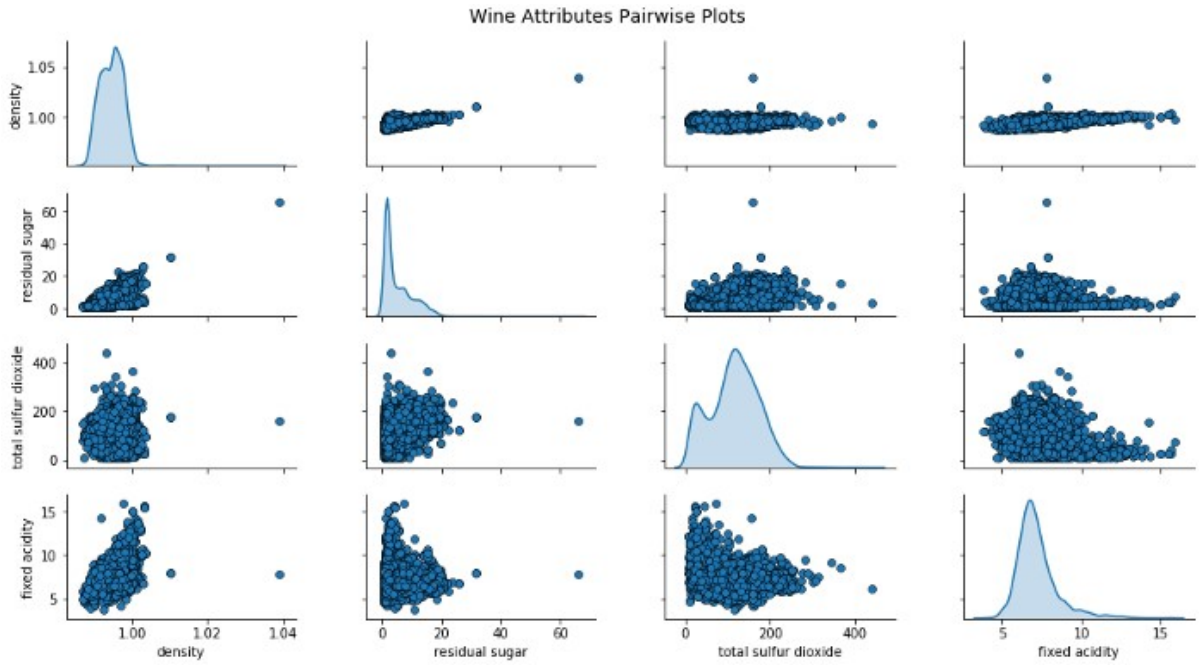
One of the best ways to check out potential relationships or correlations amongst the different data attributes is to leverage a ***pair-wise correlation matrix*** and depict it as a ***heatmap***.




**Visualizing two-dimensional data with a correlation heatmap**

The gradients in the heatmap vary based on the strength of the correlation and you can clearly see it is very easy to spot potential attributes having strong correlations amongst themselves.

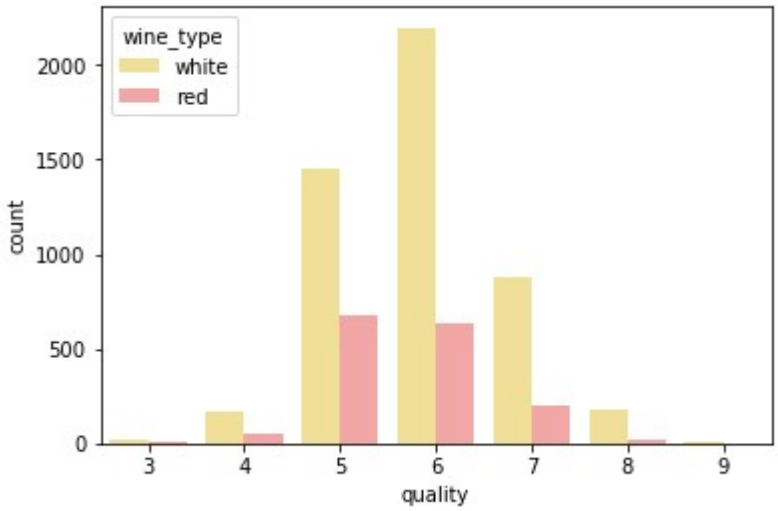
Another way to visualize the same is to use ***pair-wise scatter plots*** amongst attributes of interest.




**Visualizing two-dimensional data with pair-wise scatter plots**

Based on the above plot, you can see that scatter plots are also a decent way of observing potential relationships or patterns in two-dimensions for data attributes.

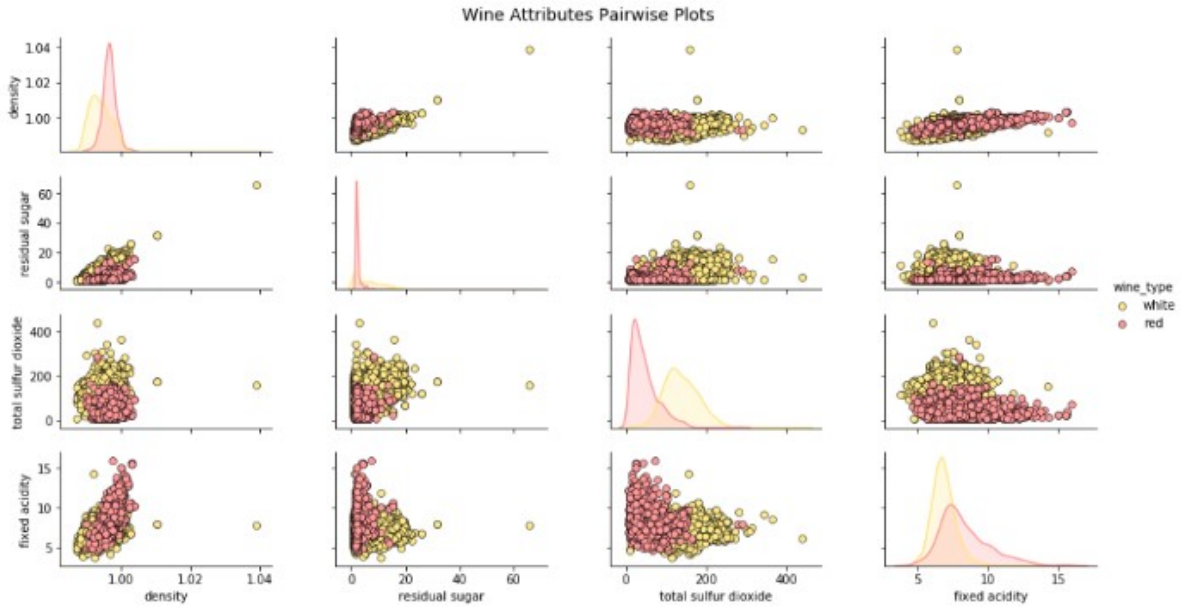
Use ***stacked bars*** or ***multiple bars*** for the different attributes in a single plot.




**Visualizing two-dimensional discrete, categorical data in a single bar chart**

## Visualizing data in Three Dimensions (3-D)

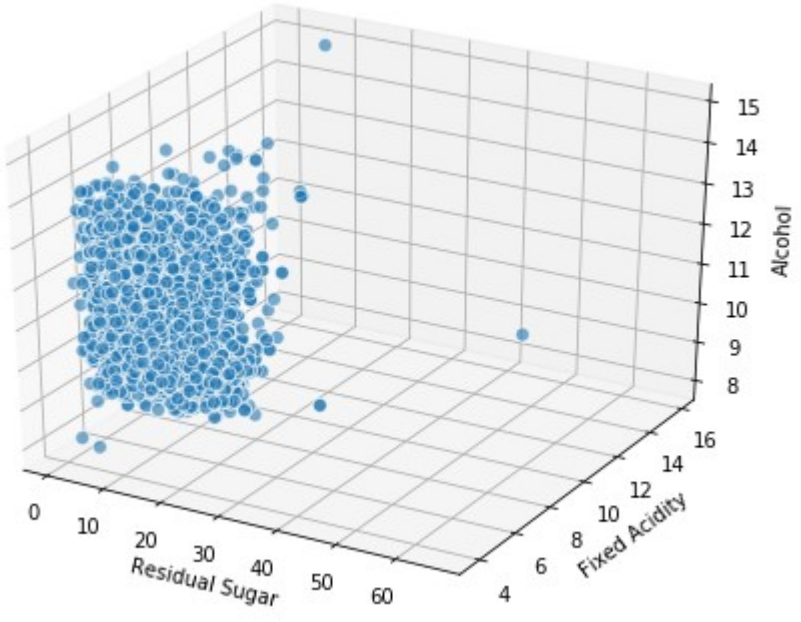
Considering three attributes or dimensions in the data, we can visualize them by considering a ***pair-wise scatter plot*** and introducing the notion of ***color*** or ***hue*** to separate out values in a categorical dimension.




**Visualizing three-dimensional data with scatter plots and** hue **(color)**

The above plot enables you to check out correlations and patterns and also compare around wine groups.

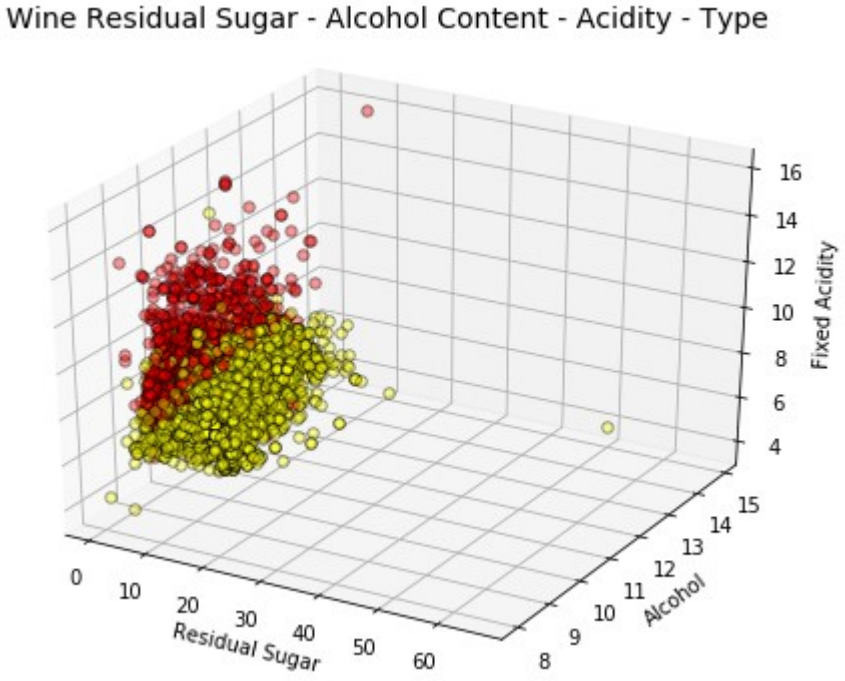
Let's look at strategies for ***visualizing three continuous, numeric attributes***. One way would be to have two dimensions represented as the regular ***length*** (*x* axis) and ***breadth*** (*y*-axis) and also take the notion of ***depth*** (*z*-axis) for the third dimension.




**Visualizing three-dimensional numeric data by introducing the notion of depth**

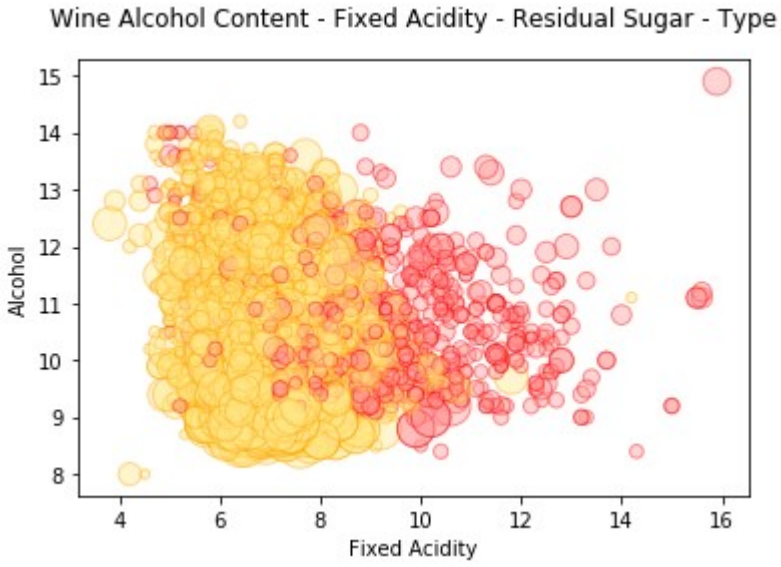
## Visualizing data in Four Dimensions (4-D)

One way to visualize data in four dimensions is to use ***depth*** and ***hue*** as specific data dimensions in a conventional plot like a ***scatter plot***.




**Visualizing data in four-dimensions leveraging scatter plots and the concept of** hue **and** depth

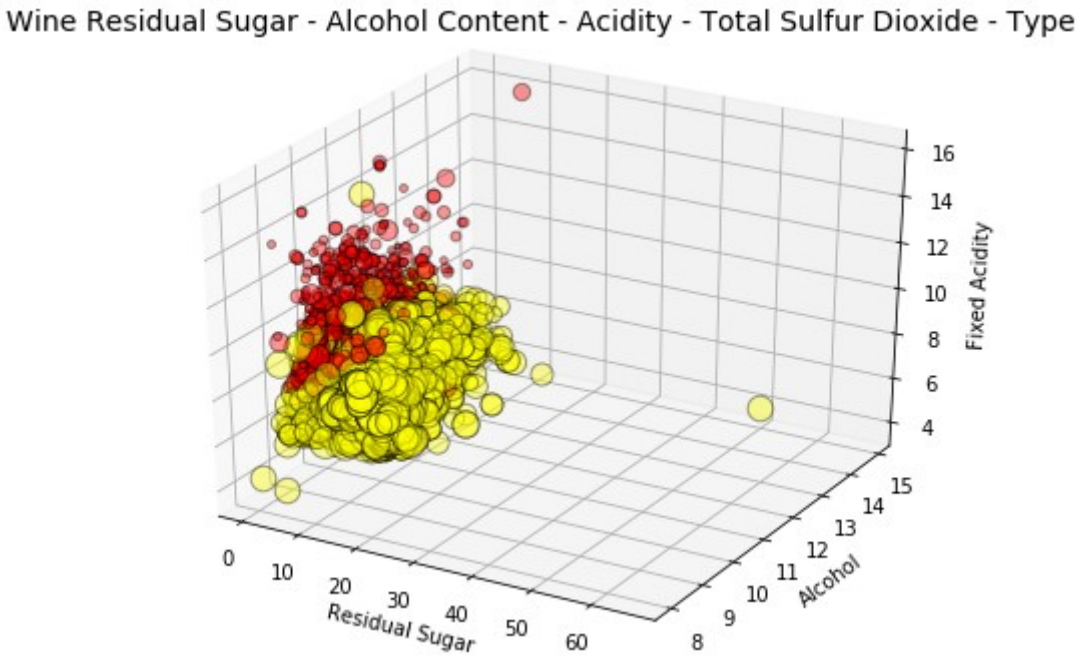
One strategy to make this better, is to keep a 2-D plot, but use ***hue*** and data point ***size*** as data dimensions. Typically this would be a ***bubble chart*** similar to what we visualized earlier.




**Visualizing data in four-dimensions leveraging bubble charts and the concept of** hue **and** size

## Visualizing data in Five Dimensions (5-D)

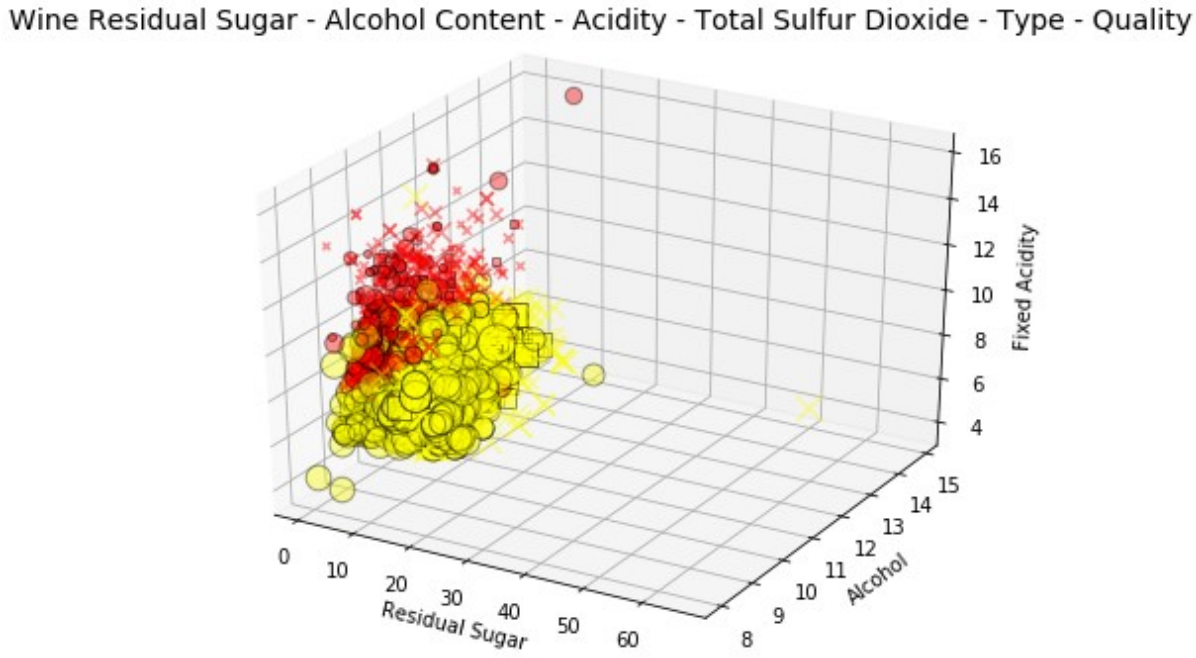
To visualize data in five dimensions, we leverage various plotting components. Let's use ***depth***, ***hue*** and ***size*** to represent three of the data dimensions besides ***regular axes*** representing the other two dimensions. Since we use the notion of size, we will be basically plotting a three dimensional ***bubble chart***.




**Visualizing data in five-dimensions leveraging bubble charts and the concept of** hue, depth **and** size

## Visualizing data in Six Dimensions (6-D)

Leverage ***depth***, ***hue***, ***size*** and ***shape*** besides our ***regular two axes*** to depict all the six data dimensions




**Visualizing data in six-dimensions leveraging scatter charts and the concept of** hue, depth, shape **and** size

## Can we go higher?

It definitely becomes more and more difficult to hack our way around the limitations of a two-dimensional rendering device to visualize more data dimensions.

## Visualizing Image Data

Images are basically multi-dimensional tensors which can be represented as matrices of pixel values. This really increases the dimensionality of even a simple small image and makes it difficult to work with them

So when and why would you want to visualize your data in a low dimension? When working on data with more than 2–3 features you might want to check if your data has clusters in it. This information can help you understand your data and, if needed, choose the number of clusters for clustering models such as k-means.

Now let's look at a short example that will help understand what we want to get. Let's say we have data in a 2D space and we want to reduce its dimension into 1D. Here's an example of data in 2D:

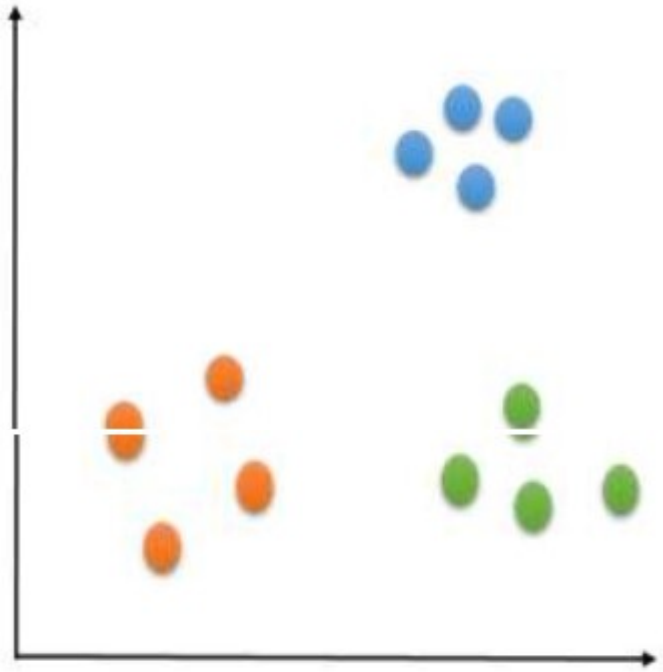


Figure 1: Original Data points in the high-dimensional space

In this example, each color represents a cluster. We can see that each cluster has a different density. We will see how the model deals with that in the dimensional reduction process.

Now, if we try to simply project the data onto just one of its dimensions, we see an overlap of at least two of the clusters:

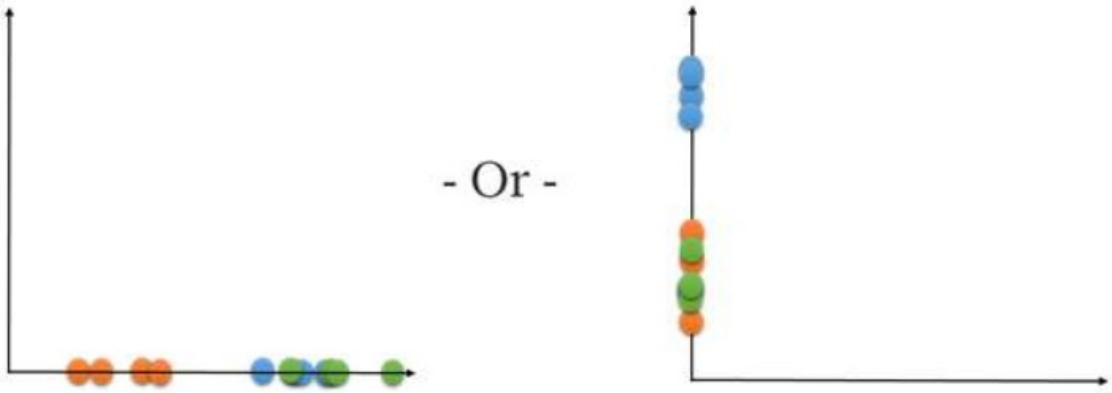



Figure 2: Data projections to one dimension

we need to find a better way to do this dimension reduction.

T-SNE algorithm deals with this problem

**What is t-SNE?**

t-Distributed Stochastic Neighbor Embedding (t-SNE) is an unsupervised, non-linear technique primarily used for data exploration and visualizing high-dimensional data. In simpler terms, t-SNE gives you a feel or intuition of how the data is arranged in a high-dimensional space. It was developed by Laurens van der Maatens and Geoffrey Hinton in 2008.

- Calculating a joint probability distribution that represents the similarities between the data points

- Creating a dataset of points in the target dimension and then calculating the joint probability distribution for them as well.
- Using gradient descent to change the dataset in the low-dimensional space so that the joint probability distribution representing it would be as similar as possible to the one in the high dimension.

# The Algorithm

## First Stage __ points, how likely they are to be neighbors

The first stage of the algorithm is calculating the Euclidian distances of each point from all of the other points. Then, taking these distances and transforming them into conditional probabilities that represent the similarity between every two points. What does that mean? It means that we want to evaluate how similar every two points in the data are, or in other words, **how likely they are to be neighbors.**

The conditional probability of point $x_j$ to be next to point $x_i$ is represented by a Gaussian centered at $x_i$ with a standard deviation of $\sigma_i$ (I'll mention later on what influences $\sigma_i$). It is written mathematically in the following way:

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}$$

The probability of point $x_i$ to have $x_j$ as it's neighbor

The reason for dividing by the sum of all the other points placed at the Gaussian centered at $x_i$ is that we may need to deal with clusters with different densities. To explain that, let's go back to the example of Figure 1. As you can see the density of the orange cluster is lower than the density of the blue cluster. Therefore, if we calculate the similarities of each two points by a Gaussian only, we will see lower similarities between the orange points compared to the blue ones. In our final output we won't mind that some clusters had different densities, we will just want to see them as clusters, and therefore we do this normalization.

From the conditional distributions created we calculate the joint probability distribution, using the following equation:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

Using the joint probability distribution rather than the conditional probability is one of the improvements in the method of t-SNE relative to the former SNE. The symmetric property of the pairwise similarities ($p_{ij} = p_{ji}$) helps simplify the calculation at the third stage of the algorithm.

## Second Stage — Creating data in a low dimension

In this stage, we create a dataset of points in a low-dimensional space and calculate a joint probability distribution for them as well.

To do that, we build a random dataset of points with the same number of points as we had in the original dataset, and K features, where K is our target dimension. Usually, K will be 2 or 3 if we want to use the dimension reduction for visualization. If we go back to our example, at this stage the algorithm builds a random dataset of points in 1D:

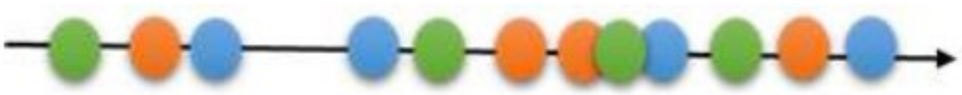


Figure 3: A random set of points in 1D

For this set of points, we will create their joint probability distribution but this time we will be using the t-distribution and not the Gaussian distribution, as we did for the original dataset. This is another advantage of t-SNE compared to the former SNE (t in t-SNE stands for t-distribution) that I will soon explain. We will mark the probabilities here by q, and the points by y.

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}}$$

The reason for choosing t-distribution rather than the Gaussian distribution is the heavy tails property of the t-distribution. This quality causes moderate distances between points in the high-dimensional space to become extreme in the low-dimensional space, and that helps prevent "crowding" of the points in the lower dimension. Another advantage of using t-distribution is an improvement in the optimization process in the third part of the algorithm.

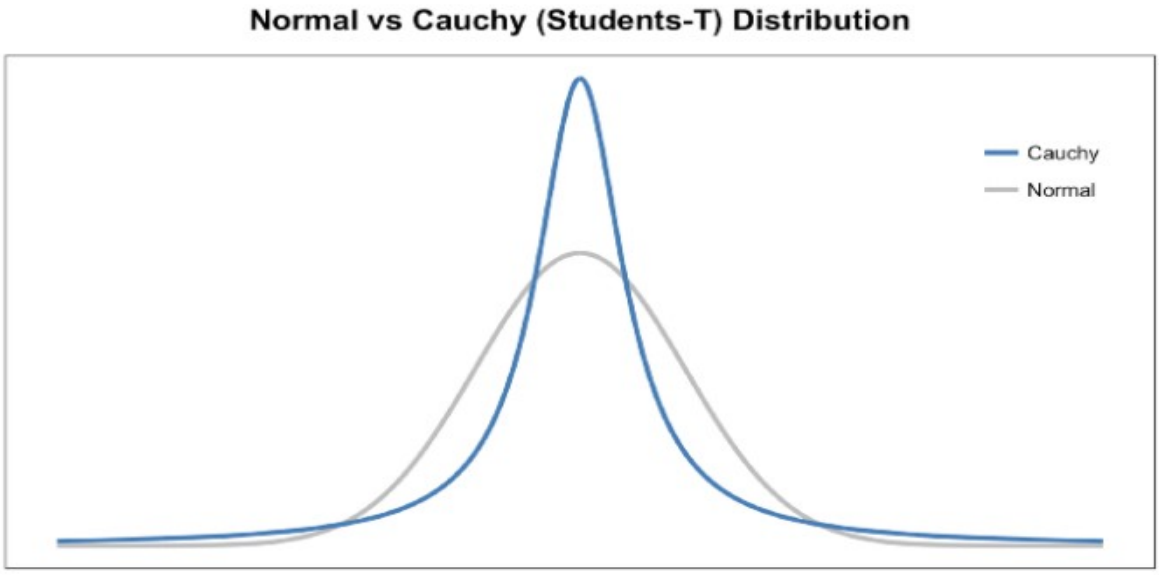



Figure 3 — Normal vs Student t-distribution

## Third Stage —

*change your dataset in the low-dimensional space so it will best visualize your data*

Now we use the Kullback-Leiber divergence to make the joint probability distribution of the data points in the low dimension as similar as possible to the one from the original dataset. If this transformation succeeds we will get a good dimension reduction.

KL divergence is
a measure of how much two distributions are different from one another.

distributions P and Q in the probability space of $\chi$, the KL divergence is defined by:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log\left(\frac{P(x)}{Q(x)}\right)$$

The definition of KL divergence between the probability distributions P and Q

As much as the distributions are similar to each other, the value of the KL divergence is smaller, reaching zero when the distributions are identical.

Back to our algorithm — we try to change the lower dimension dataset such that its joint probability distribution will be as similar as possible to the one from the original data. This is done by using gradient descent. The cost function that the gradient descent tries to minimize is the KL divergence of the joint probability distribution P from the high-dimensional space and Q from the low-dimensional space.

$$C = KL(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

The cost function for the gradient descent is the KL divergence between P and Q, the joint probability distributions of the high and low dimensions respectively

From this optimization, we get the values of the points in the low dimension dataset and use it for our visualization. In our example, we see the clusters in the low-dimensional space as follows:



## Parameters in the model

There are several parameters in this model that you can adjust to your needs. Some of them relate to the process of gradient descent, where the most important ones are the learning rate and the number of iterations.

Another parameter in t-SNE is **perplexity**. It is used for choosing the standard deviation $\sigma_i$ of the Gaussian representing the conditional distribution in the high-dimensional space. I will not elaborate on the math behind it, but it can be interpreted as the number of effective neighbors for each point. The model is rather robust for

perplexities between 5 to 50

**Algorithm 1**: Simple version of t-Distributed Stochastic Neighbor Embedding.

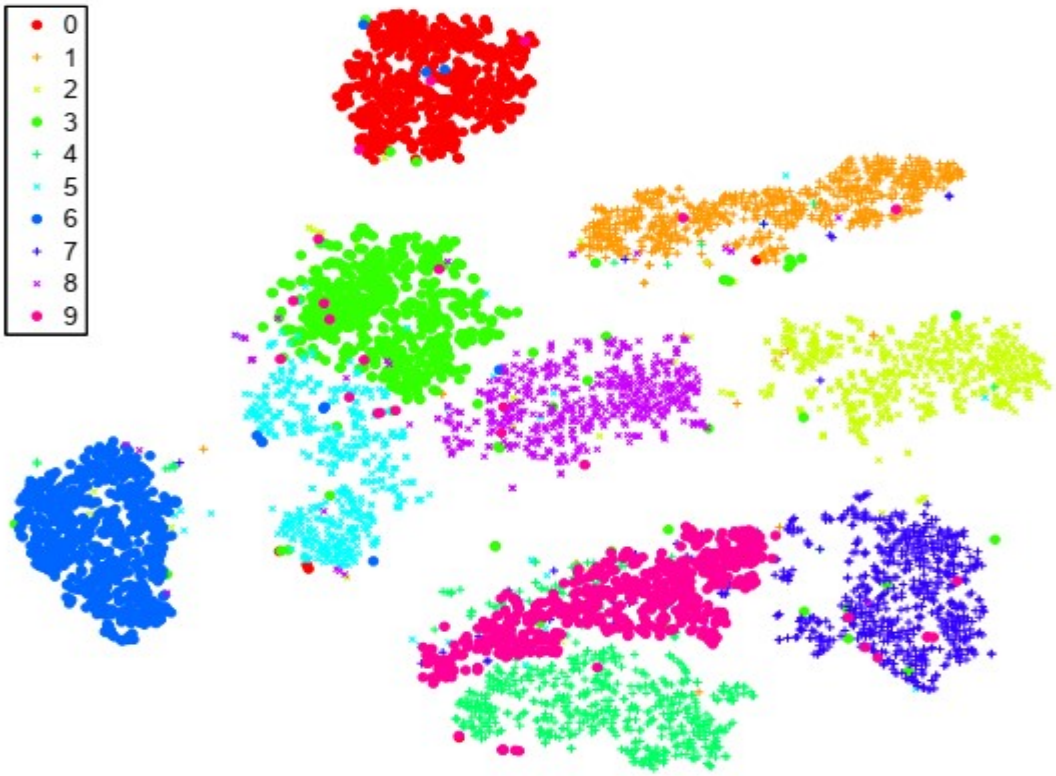
**Data**: data set $\mathcal{X} = \{x_1, x_2, ..., x_n\}$,
cost function parameters: perplexity *Perp*,
optimization parameters: number of iterations $T$, learning rate $\eta$, momentum $\alpha(t)$.
**Result**: low-dimensional data representation $\mathcal{Y}^{(T)} = \{y_1, y_2, ..., y_n\}$.
**begin**
  compute pairwise affinities $p_{j|i}$ with perplexity *Perp* (using Equation 1)
  set $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$
  sample initial solution $\mathcal{Y}^{(0)} = \{y_1, y_2, ..., y_n\}$ from $\mathcal{N}(0, 10^{-4}I)$
  **for** $t$=1 **to** $T$ **do**
    compute low-dimensional affinities $q_{ij}$ (using Equation 4)
    compute gradient $\frac{\delta C}{\delta \mathcal{Y}}$ (using Equation 5)
    set $\mathcal{Y}^{(t)} = \mathcal{Y}^{(t-1)} + \eta \frac{\delta C}{\delta \mathcal{Y}} + \alpha(t)\left(\mathcal{Y}^{(t-1)} - \mathcal{Y}^{(t-2)}\right)$
  **end**
**end**

The MNIST data set[6] contains 60,000 grayscale images of handwritten digits. For our experiments, we randomly selected 6,000 of the images for computational reasons. The digit images have $28 \times 28 = 784$ pixels (i.e., dimensions). The Olivetti faces data set[7] consists of images of 40 individuals with small variations in viewpoint, large variations in expression, and occasional addition of glasses. The data set consists of 400 images (10 per individual) of size $92 \times 112 = 10,304$ pixels, and is labeled according to identity. The COIL-20 data set (Nene et al., 1996) contains images of 20 different objects viewed from 72 equally spaced orientations, yielding a total of 1,440 images. The images contain $32 \times 32 = 1,024$ pixels.

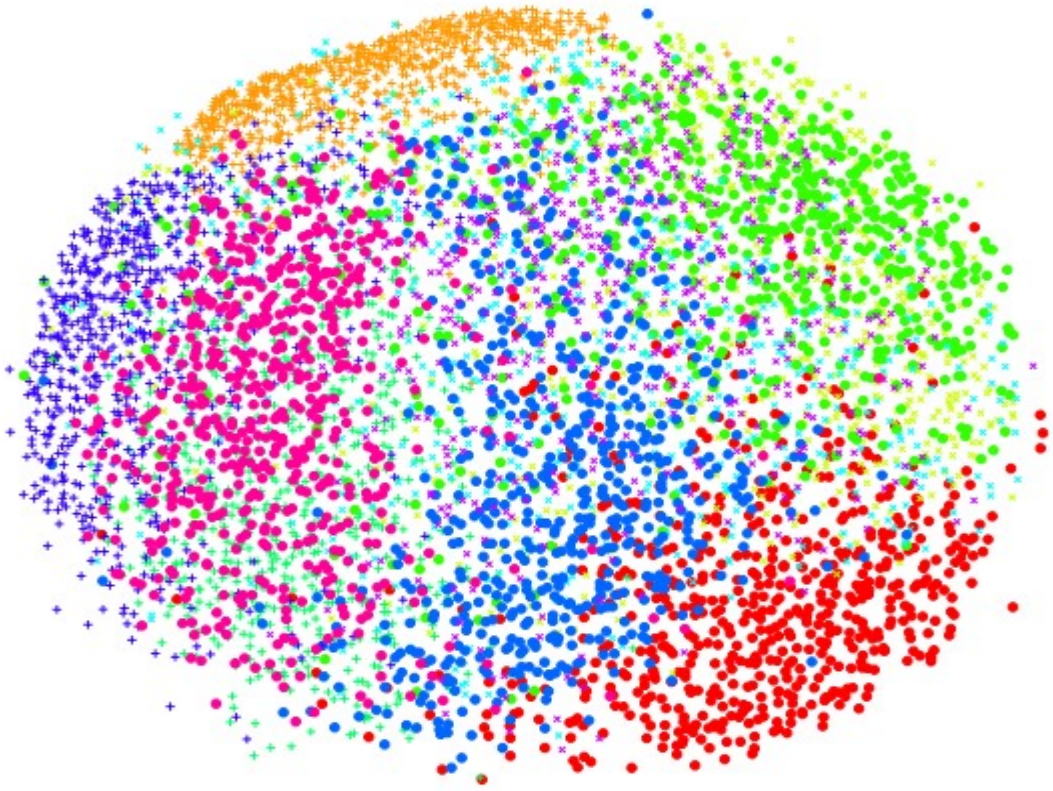
| *Technique* | *Cost function parameters* |
|---|---|
| t-SNE | $Perp = 40$ |
| Sammon mapping | none |
| Isomap | $k = 12$ |
| LLE | $k = 12$ |

Table 1: Cost function parameter settings for the experiments.

*Perp* represents the perplexity of the conditional probability distribution induced by a Gaussian kernel and $k$ represents the number of nearest neighbors employed in a neighborhood graph.




(a) Visualization by t-SNE.



(b) Visualization by Sammon mapping.

Figure 2: Visualizations of 6,000 handwritten digits from the MNIST data set.

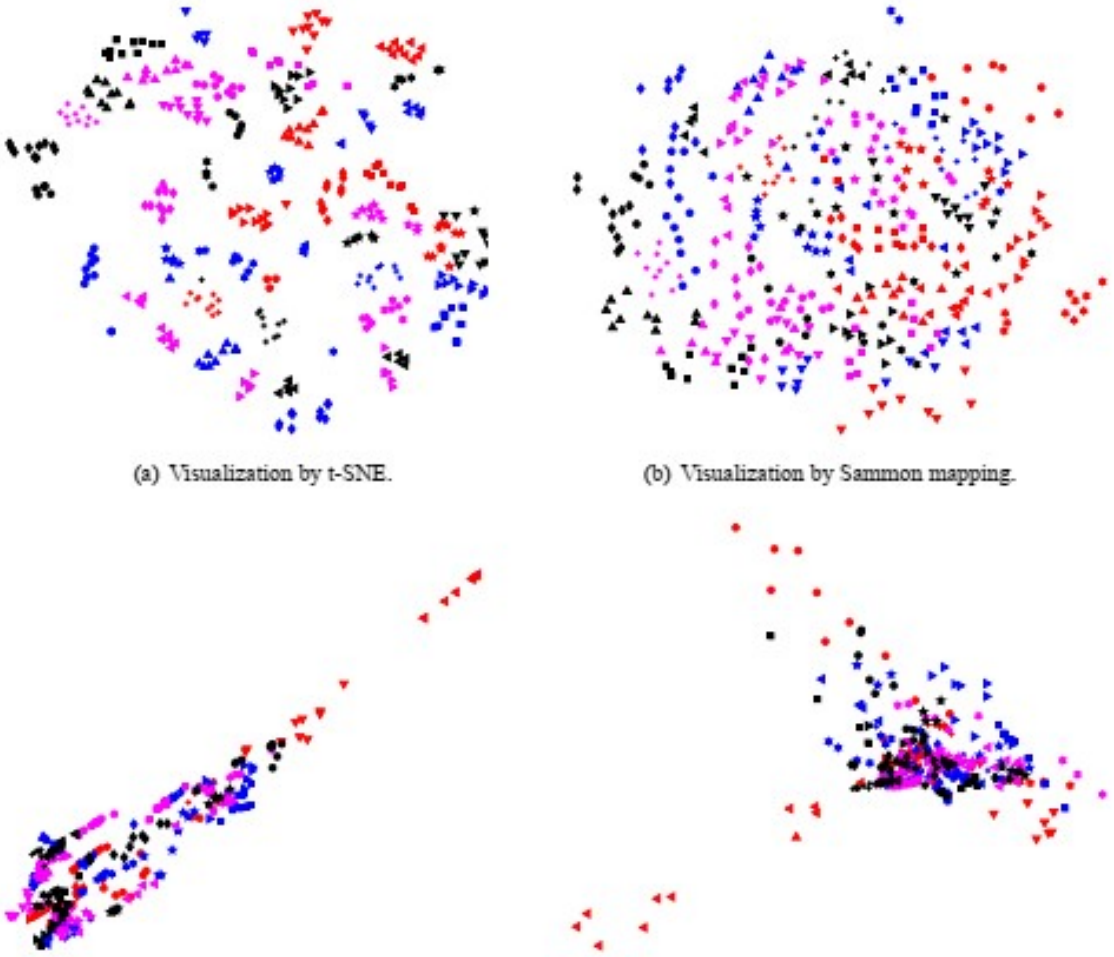


(a) Visualization by Isomap.



(b) Visualization by LLE.

Figure 3: Visualizations of 6,000 handwritten digits from the MNIST data set.



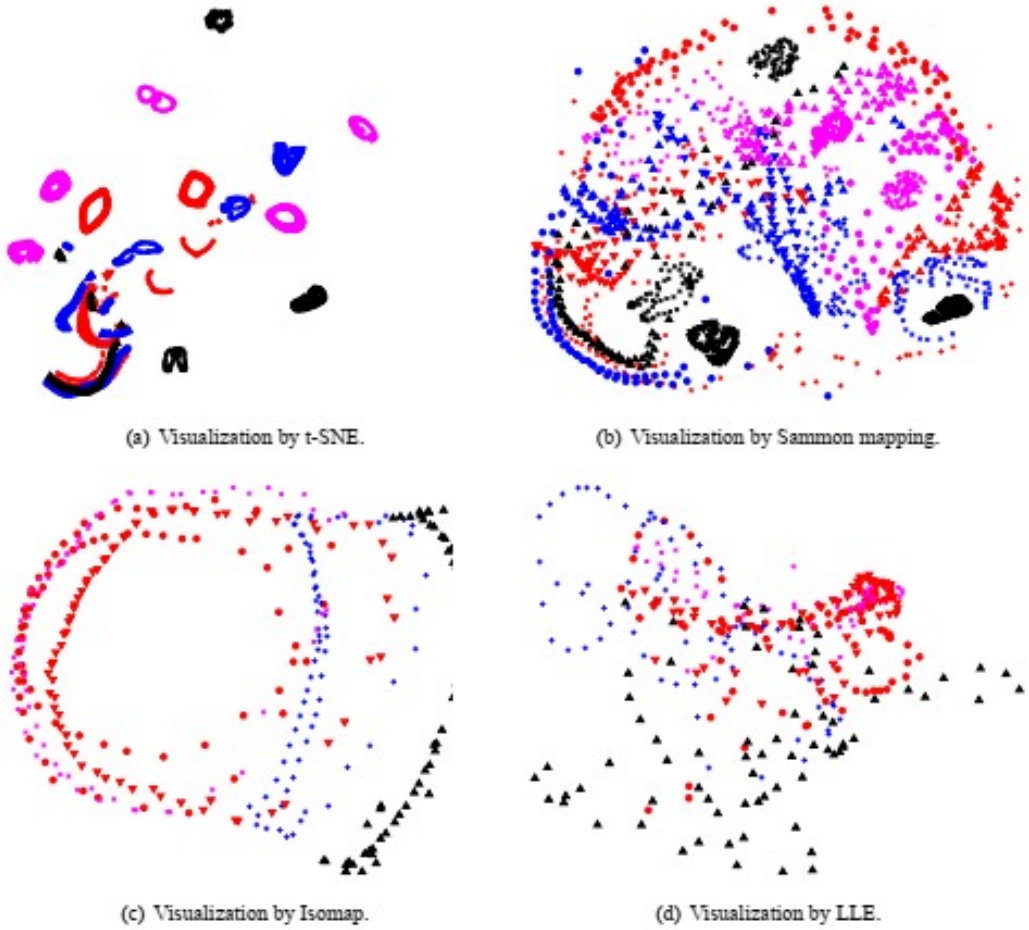
(a) Visualization by t-SNE.

(b) Visualization by Sammon mapping.

(c) Visualization by Isomap.

(d) Visualization by LLE.

Figure 4: Visualizations of the Olivetti faces data set.



(a) Visualization by t-SNE.

(b) Visualization by Sammon mapping.

(c) Visualization by Isomap.

(d) Visualization by LLE.

Figure 5: Visualizations of the COIL-20 data set.

## difference between PCA and t-SNE.

PCA is a linear dimension reduction technique that seeks to maximize variance and preserves large pairwise distances. In other words, things that are different end up far apart. This can lead to poor visualization especially when dealing with non-linear manifold structures. Think of a manifold structure as any geometric shape like: cylinder, ball, curve, etc.

t-SNE differs from PCA by preserving only small pairwise distances or local similarities whereas PCA is concerned with preserving large pairwise distances to maximize variance. Laurens illustrates the PCA and t-SNE approach pretty well using the Swiss Roll dataset in Figure 1 [1]. You can see that due to the non-linearity of this toy dataset (manifold) and preserving large distances that PCA would incorrectly preserve the structure of the data.

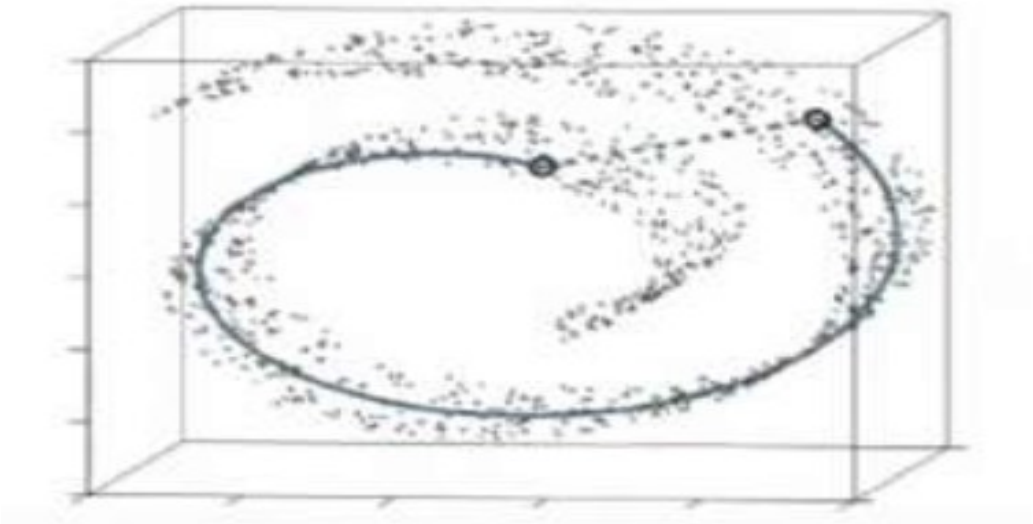


Figure 1 — Swiss Roll Dataset. Preserve small distance with t-SNE (solid line) vs maximizing variance PCA [1]

## How t-SNE works

The t-SNE algorithm calculates a similarity measure between pairs of instances in the high dimensional space and in the low dimensional space. It then tries to optimize these two similarity measures using a cost function. Let's break that down into 3 basic steps.

1. Step 1, measure similarities between points in the high dimensional space. Think of a bunch of data points scattered on a 2D space (Figure 2). For each data point (xi) we'll center a Gaussian distribution over that point. Then we measure the density of all points (xj) under that Gaussian distribution. Then renormalize for all points. This gives us a set of probabilities (Pij) for all points. Those probabilities are proportional to the similarities. All that means is, if data points x1 and x2 have equal values under this

distribution or circle can be manipulated using what's called perplexity, which influences the variance of the distribution (circle size) and essentially the number of nearest neighbors. Normal range for perplexity is between 5 and 50 [2].

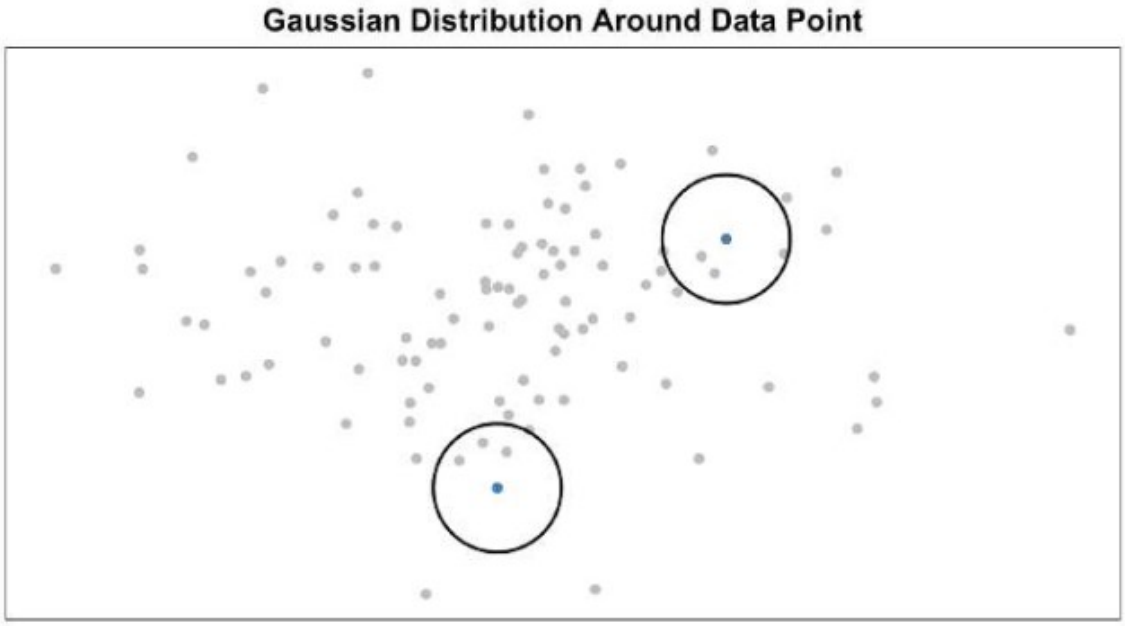



Figure 2 — Measuring pairwise similarities in the high-dimensional space

2. Step 2 is similar to step 1, but instead of using a Gaussian distribution you use a Student t-distribution with one degree of freedom, which is also known as the Cauchy distribution (Figure 3). This gives us a second set of probabilities (Qij) in the low dimensional space. As you can see the Student t-distribution has heavier tails than the normal distribution. The heavy tails allow for better modeling of far apart distances.

3. The last step is that we want these set of probabilities from the low-dimensional space (Qij) to reflect those of the high dimensional space (Pij) as best as possible. We want the two map structures to be similar. We measure the difference between the probability distributions of the two-dimensional spaces using Kullback-Liebler divergence (KL). I won't get too much into KL except that it is an asymmetrical approach that efficiently compares large Pij and Qij values. Finally, we use gradient descent to minimize our KL cost function.

## 2. Stochastic Neighbor Embedding

Stochastic Neighbor Embedding (SNE) starts by converting the high-dimensional Euclidean distances between datapoints into conditional probabilities that represent similarities.[1] The similarity of datapoint $x_j$ to datapoint $x_i$ is the conditional probability, $p_{j|i}$, that $x_i$ would pick $x_j$ as its neighbor if neighbors were picked in proportion to their probability density under a Gaussian centered at $x_i$. For nearby datapoints, $p_{j|i}$ is relatively high, whereas for widely separated datapoints, $p_{j|i}$ will be almost infinitesimal (for reasonable values of the variance of the Gaussian, $\sigma_i$). Mathematically, the conditional probability $p_{j|i}$ is given by

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2/2\sigma_i^2\right)}{\sum_{k\neq i}\exp\left(-\|x_i - x_k\|^2/2\sigma_i^2\right)},$$

where $\sigma_i$ is the variance of the Gaussian that is centered on datapoint $x_i$. The method for determining the value of $\sigma_i$ is presented later in this section. Because we are only interested in modeling pairwise similarities, we set the value of $p_{i|i}$ to zero. For the low-dimensional counterparts $y_i$ and $y_j$ of the high-dimensional datapoints $x_i$ and $x_j$, it is possible to compute a similar conditional probability, which we denote by $q_{j|i}$. We set[2] the variance of the Gaussian that is employed in the computation of the conditional probabilities $q_{j|i}$ to $\frac{1}{\sqrt{2}}$. Hence, we model the similarity of map point $y_j$ to map point $y_i$ by

$$q_{j|i} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k\neq i}\exp\left(-\|y_i - y_k\|^2\right)}.$$

Again, since we are only interested in modeling pairwise similarities, we set $q_{i|i} = 0$.

If the map points $y_i$ and $y_j$ correctly model the similarity between the high-dimensional datapoints $x_i$ and $x_j$, the conditional probabilities $p_{j|i}$ and $q_{j|i}$ will be equal. Motivated by this observation, SNE aims to find a low-dimensional data representation that minimizes the mismatch between $p_{j|i}$ and $q_{j|i}$. A natural measure of the faithfulness with which $q_{j|i}$ models $p_{j|i}$ is the Kullback-Leibler divergence (which is in this case equal to the cross-entropy up to an additive constant). SNE minimizes the sum of Kullback-Leibler divergences over all datapoints using a gradient descent method. The cost function $C$ is given by

$$C = \sum_i KL(P_i||Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}, \qquad (2)$$

in which $P_i$ represents the conditional probability distribution over all other datapoints given datapoint $x_i$, and $Q_i$ represents the conditional probability distribution over all other map points given map point $y_i$. Because the Kullback-Leibler divergence is not symmetric, different types of error in the pairwise distances in the low-dimensional map are not weighted equally. In particular, there is a large cost for using widely separated map points to represent nearby datapoints (i.e., for using a small $q_{j|i}$ to model a large $p_{j|i}$), but there is only a small cost for using nearby map points to represent widely separated datapoints. This small cost comes from wasting some of the probability mass in the relevant $Q$ distributions. In other words, the SNE cost function focuses on retaining the local structure of the data in the map (for reasonable values of the variance of the Gaussian in the high-dimensional space, $\sigma_i$).

The remaining parameter to be selected is the variance $\sigma_i$ of the Gaussian that is centered over each high-dimensional datapoint, $x_i$. It is not likely that there is a single value of $\sigma_i$ that is optimal for all datapoints in the data set because the density of the data is likely to vary. In dense regions, a smaller value of $\sigma_i$ is usually more appropriate than in sparser regions. Any particular value of $\sigma_i$ induces a probability distribution, $P_i$, over all of the other datapoints. This distribution has an entropy which increases as $\sigma_i$ increases. SNE performs a binary search for the value of $\sigma_i$ that produces a $P_i$ with a fixed perplexity that is specified by the user.[3] The perplexity is defined as

$$Perp(P_i) = 2^{H(P_i)},$$

where $H(P_i)$ is the Shannon entropy of $P_i$ measured in bits

$$H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}.$$

The perplexity can be interpreted as a smooth measure of the effective number of neighbors. The performance of SNE is fairly robust to changes in the perplexity, and typical values are between 5 and 50.

The minimization of the cost function in Equation 2 is performed using a gradient descent method. The gradient has a surprisingly simple form

$$\frac{\delta C}{\delta y_i} = 2\sum_j (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})(y_i - y_j).$$

## t-Distributed Stochastic Neighbor Embedding

SNE con-

structs reasonably good visualizations, it is hampered by a cost function that is difficult to optimize and by a problem we refer to as the "crowding problem". In this section, we present a new technique called "t-Distributed Stochastic Neighbor Embedding" or "t-SNE" that aims to alleviate these problems. The cost function used by t-SNE differs from the one used by SNE in two ways: (1) it uses a symmetrized version of the SNE cost function with simpler gradients that was briefly introduced by Cook et al. (2007) and (2) it uses a Student-t distribution rather than a Gaussian to compute the similarity between two points *in the low-dimensional space*. t-SNE employs a heavy-tailed distribution in the low-dimensional space to alleviate both the crowding problem and the optimization problems of SNE.